National University of Computer and Emerging Sciences
Islamabad Operating System Lab Spring 2025

Lab Task 12

Lab Task 01: Concurrent Inventory Updater (Mutex Protection)

Objective:
Use mutex to ensure safe and synchronized updates to a warehouse inventory by multiple restocking and purchasing threads.

Scenario:
A warehouse has a shared inventory of 100 items. Multiple threads represent either a **restocking agent** (adding stock) or a **customer purchase** (removing stock). All updates to inventory must be synchronized to prevent race conditions and ensure inventory never goes negative.

Instructions:

- Declare a shared variable `inventory = 100`.
- Create a total of 6 threads:
    - 3 threads simulate customer purchases (each trying to purchase 30 items).
    - 3 threads simulate restocking agents (each adding 20 items).
- Each thread:
    - Locks the mutex before accessing the inventory.
    - Updates the inventory based on its role.

    - Unlocks the mutex after updating.
- Display messages for each thread showing:
    - Action (purchase or restock)
    - Items added or removed
    - Remaining inventory
- Ensure the final inventory is correctly calculated and non-negative.

Lab Task 02: Multi-User Ticket Booking System (Mutex Protection)

Objective:
Implement a ticket booking system that handles concurrent user requests safely using mutex.

Scenario:
A ticket counter has 5 tickets available. Multiple users try to book tickets at the same time. Only one user should be able to book a ticket at a time, and the counter should not go negative.

Instructions:

- Declare a shared variable `availableTickets = 5`.
- Create 6 threads representing different users.
- Each thread checks if a ticket is available and books it if so.
- Protect the ticket booking section using `pthread_mutex_lock()` and `pthread_mutex_unlock()`.
- Print appropriate messages for booking success or failure.

Lab Task 03: Parallel Matrix Row Sum with Mutex

Objective:
Safely calculate the sum of rows in a matrix using multithreading and mutex to update a shared result.

Scenario:
Given a 3x3 matrix, each thread computes the sum of one row and adds it to a global result variable.

Instructions:

- Define a 3x3 matrix with predefined values.
- Create one thread per row.
- Each thread calculates the sum of its row and adds it to a shared `totalSum`.
- Use mutex to synchronize access to `totalSum`.
- Display each row sum and final matrix sum in the main thread.

Lab Task 04: Concurrent Circle Area Calculator (trylock usage)

Objective:
Use `pthread_mutex_trylock()` to simulate concurrent area calculations where only one thread can access the shared formula resource at a time.

Scenario:
Multiple threads attempt to calculate the area of a circle using different radii. However, due to a limited computation unit, only one thread can access the calculation at a time. If the unit is busy, threads skip that cycle.

Instructions:

- Create 4 threads, each with a unique radius.
- Each thread attempts to acquire the mutex using `pthread_mutex_trylock()`.
- If successful:
    - Calculate area using `πr²`.
    - Add to shared `totalArea`.
    - Unlock the mutex.
- If the lock is unavailable, print `"Resource busy. Skipping this calculation."`
- Print each successful calculation and the final `totalArea`.

THE END
